Azimi Innovation Lab
AI Architect · End-to-End Projects

جلسه 9

گیت و گیت‌هاب عمیق‌تر

شاخه‌ها، کانفلیکت و پول‌ریکوئست

ادامه مستقیم جلسات قبل — Azimi Innovation Lab

Azimi Innovation Lab
AI Architect · End-to-End Projects

اتصال به جلسات قبل: شما قبلاً چرخه انت‌تواند را انجام داده‌اید و کد را روی گیت‌هاب گذاشته‌اید. حالا که عمق فنی بیشتری دارید (خواندن کد، تست، API، دیتابیس)، وقت آن است که گیت را عمیق‌تر بفهمید تا بتوانید تغییرات را بهتر مدیریت کنید، اشتباهات را برگردانید و با خیال راحت‌تر کار کنید.

۱. یادآوری سریع: گیت و گیت‌هاب

گیت سیستم کنترل نسخه است که تاریخچه تغییرات کد را نگه می‌دارد. گیت‌هاب جایی است که این تاریخچه را آنلاین ذخیره و به اشتراک می‌گذاریم.

- **Repository** (ریپازیتوری): پوشه پروژه به همراه تاریخچه آن.
- **Commit**: یک نقطه ذخیره از تغییرات با پیام توضیحی.
- **Push**: فرستادن کامیت‌ها به گیت‌هاب.
- **Pull**: گرفتن آخرین تغییرات از گیت‌هاب.

۲. شاخه‌ها (Branches)

شاخه یک خط جداگانه از توسعه است. شاخه اصلی معمولاً main یا master نام دارد. وقتی می‌خواهید ویژگی جدیدی اضافه کنید یا چیزی را آزمایش کنید، یک شاخه جدید می‌سازید تا شاخه اصلی خراب نشود.

تشبیه: شاخه اصلی مثل نسخه نهایی یک کتاب است. وقتی می‌خواهید فصل جدیدی بنویسید، یک کپی از کتاب برمی‌دارید (شاخه جدید)، تغییرات را آنجا می‌دهید و وقتی مطمئن شدید، آن را به کتاب اصلی اضافه می‌کنید.

```
# ساخت شاخه جدید
git checkout -b feature/new-button

# یا با دستور جدیدتر
git switch -c feature/new-button

# دیدن شاخه‌ها
git branch
```

Azimi Innovation Lab
AI Architect · End-to-End Projects

۳. پول‌ریکوئست (Pull Request یا PR)

وقتی کار روی یک شاخه تمام شد، یک Pull Request می‌سازید تا تغییرات را به شاخه اصلی پیشنهاد دهید. در گیت‌هاب می‌توانید تغییرات را ببینید، بحث کنید و سپس آن را merge (ادغام) کنید.

1. شاخه را push کنید.
2. در گیت‌هاب روی Compare & pull request کلیک کنید.
3. توضیح بنویسید که چه تغییری داده‌اید.
4. بعد از بررسی، Merge کنید.

چرا مهم است؟ حتی اگر تنها کار می‌کنید، PR به شما کمک می‌کند قبل از ادغام، تغییرات را یک‌بار دیگر مرور کنید و تاریخچه تمیزتری داشته باشید.

۴. کانفلیکت (Conflict)

وقتی دو نفر (یا دو شاخه) یک خط کد را به شکل متفاوتی تغییر داده باشند، گیت نمی‌تواند خودکار تصمیم بگیرد و کانفلیکت ایجاد می‌شود.

```
<<<<<<< HEAD
متن نسخه فعلی
=======
متن نسخه دیگر
>>>>>>> feature-branch
```

شما باید تصمیم بگیرید کدام نسخه درست است (یا هر دو را ترکیب کنید)، علامت‌های کانفلیکت را حذف کنید و سپس کامیت کنید.

۵. دستورات مفید روزمره

```
# وضعیت فعلی
git status

# اضافه کردن فایل‌ها
git add .

# کامیت با پیام
git commit -m "اضافه کردن دکمه تغییر رنگ"
```

```
# فرستادن به گیت‌هاب
git push

# گرفتن آخرین تغییرات
git pull

# دیدن تاریخچه
git log --oneline
```

۶. اشتباهات رایج و راه حل

- فراموش کردن کامیت: قبل از تغییر بزرگ، یک کامیت بگیرید تا بتوانید برگردید.
- پیام کامیت مبهم: پیام را واضح بنویسید (مثلاً «رفع باگ نمایش لیست» بهتر از «تغییرات»).
- کار مستقیم روی **main**: برای تغییرات مهم از شاخه استفاده کنید.
- ترس از کانفلیکت: کانفلیکت عادی است. با آرامش علامت‌ها را بخوانید و تصمیم بگیرید.

۷. تمرین‌های عملی

تمرین ۱: ساخت شاخه

در یکی از پروژه‌های قبلی‌تان یک شاخه جدید بسازید، یک تغییر کوچک بدهید (مثلاً تغییر یک متن)، کامیت کنید و push کنید.

تمرین ۲: ساخت Pull Request

برای همان شاخه یک Pull Request بسازید. توضیحات بنویسید و سپس آن را merge کنید.

تمرین ۳: مشاهده تاریخچه

با دستور git log --oneline تاریخچه پروژه را ببینید و حداقل سه کامیت آخر را بخوانید.

۸. چک‌لیست پایان جلسه

- می‌دانم شاخه چیست و چرا از آن استفاده می‌کنیم.

- می‌توانم یک شاخه جدید بسازم و تغییرات را push کنم.
- می‌دانم Pull Request چیست.
- مفهوم کانفلیکت را فهمیده‌ام.
- تمرین‌ها را انجام داده‌ام.

۹. خلاصه و پل به جلسه بعد

در این جلسه گیت را عمیق‌تر فهمیدید: شاخه‌ها برای کار ایمن، Pull Request برای مرور تغییرات و کانفلیکت برای حل اختلاف نسخه‌ها. حالا می‌توانید با اطمینان بیشتری کدهای خود را مدیریت کنید.

جلسه بعد: دیپلوی و محیط‌های مختلف (لوکال، استیجینگ، پروداکشن).

Azimi Innovation Lab
AI Architect · End-to-End Projects

پایان جلسه 9 — Azimi Innovation Lab

حالا مدیریت نسخه‌ها را بهتر در دست دارید. موفق باشید!